



Eduardo Dragone Pinheiro Corrêa(eduardo.correa@ucsal.edu.br) Artur de

Almeida Ramos(artur.ramos@ucsal.edu.br)

Miguel Antonio Torres Diogo(miguel.diogo@ ucsal.edu.br)

Anthony Davidson Bitencourt Moura de Araújo(anthony.araujo @ucsal.edu.br)

Docente: Pedro Arthur de Melo Nascimento(pedroam.nascimento@pro.ucsal.br)

Relatório Pokesal – Real Madrid dos Códigos

Sistema completo de batalhas Pokesals

Salvador

2026

SUMÁRIO

• 1. Informações Gerais do Projeto	3
• 2. Parte 1 — Revisão Manual de Código-Fonte (Checklist)	3
2.1 Variáveis Inicializadas Antes do Uso	3
2.2 Variáveis Declaradas e Nunca Usadas	3
2.3 Código Inacessível	3
2.4 Código Duplicado	4
2.5 Erros de Sintaxe	4
2.6 Erros de Lógica	4
2.7 Erros de Tipagem	5
2.8 Fluxo de Controle e Validade	5
2.9 Outros Tipos de Erros	5
2.10 Tabela Consolidada de Achados	5
2.11 Conclusão da Parte 1	6
• 3. Parte 2 — Análise Automatizada (SonarQube)	6
3.1 Objetivo	6
3.2 Ambiente de Análise	6
3.3 Execução	6
3.4 Resultados Mapeados	7
3.5 Problemas Encontrados — Detalhamento	8
3.6 Comparação com o TXT Exportado	8
3.7 Conclusão da Parte 2	8
• 4. Parte 3 — Síntese Consolidada	9
4.1 Pontos de Convergência entre as Análises	9
4.2 Pontos Exclusivos de Cada Abordagem	9
4.3 Recomendações Priorizadas — Backlog de Correção	10
• 5. Testes Unitários com JUnit	11
5.1 Objetivo	11
5.2 Conceitos de JUnit Utilizados no Projeto	11
5.3 Ambiente e Forma de Execução	12
5.4 Testes Existentes	12
5.4.1 BatalhaTest	12
5.4.2 TreinadorTest	13
5.5 Resultado da Execução	13
5.6 Cobertura de Código — JaCoCo	14
5.6.1 Cobertura por Classe	14
5.7 Análise Crítica dos Testes	15
5.7.1 Pontos Fortes	15
5.7.2 Pontos de Atenção	15
5.8 Lacunas de Cobertura e Testes Sugeridos	16
5.9 Configuração para Maven e SonarQube	16
5.10 Conclusão	17
• 6. Relatório de Contribuição Individual	18

1- Revisão Manual de Código-Fonte (Checklist)

Cada uma das 9 perguntas obrigatórias do checklist foi respondida com base na leitura integral do código-fonte enviado (16 arquivos .java). Ao final desta parte, uma tabela consolida todos os achados individuais em formato de lista de ação, com local exato e recomendação.

1.1. Todas as variáveis estão inicializadas antes do uso?

Resposta: Parcial — atenção necessária.

Justificativa:

- Variáveis locais: Sim. O compilador Java exige atribuição definida de variáveis locais antes do uso (definite assignment), eliminando o risco de leitura de variável local não inicializada nos 16 arquivos.
- Campos de instância: Parcialmente. Alguns campos dependem de um setter ser chamado externamente antes do uso, sem verificação em tempo de execução: `Treinador.pokesal` só é definido quando `setPokesal()` é chamado (em `Main.java` isso ocorre logo após a criação do `Treinador`, não gerando falhas no fluxo atual), e `Terreno.terreno` só é definido via `setTerreno()`.

Evidências (arquivo / trecho):

- `Treinador.java`, campo `pokesal` (linha 16) — sem valor padrão; usado em `escolhaDePerk()`, `ataquePokesal()` e nos getters antes de qualquer verificação de nulidade.
- `Terreno.java`, campo `terreno` (linha 20) — mesmo padrão (ver achado nº 8 da tabela consolidada).

1.2. Há variáveis declaradas e nunca usadas?

Resposta: Sim.

Justificativa: Não foram encontradas variáveis locais órfãs. O problema surge em nível de método/classe: há métodos e classes inteiras declarados que nunca são chamados ou instanciados a partir do ponto de entrada (`Main.java`) ou de qualquer outro ponto do código.

Evidências (arquivo / trecho):

- `Batalha.java` — método privado `escolhaItem(Treinador)` (linhas 89–98): nunca é chamado.
- `Terreno.java` — método público `efeitoFimDeTurno(Pokesal)` (linhas 91–97): nunca é chamado por `Batalha`.
- `Pokesal.java` — método `atacar(Pokesal)` (linhas 33–35): nunca é chamado.
- `items/SuperPotion.java` e `items/Antidote.java`: classes nunca instanciadas.
- `pokesal/BulbaSal.java`, `ChikoSal.java`, `CyndaSal.java`, `TotoSal.java`: classes nunca instanciadas (apenas `CharSal` e `SquirtSal` são usadas em `Main.java`).

1.3. Existe código inacessível?

Resposta: Sim.

Justificativa: Os métodos citados no item anterior são inacessíveis a partir do fluxo real de execução do programa (nenhum caminho a partir de `Main.main()` chega até eles). O compilador não os marca como "unreachable code" por serem métodos válidos que poderiam ser chamados por código externo, mas do ponto de vista funcional do sistema atual eles nunca são exercitados. O caso mais relevante é `Terreno.efeitoFimDeTurno()`, cuja ausência de chamada anula uma regra de negócio documentada (cura de terreno para `PokeSal` do tipo `Planta` no `Canteiro Central`).

Evidências (arquivo / trecho):

- `Batalha.escolhaItem()` — inacessível a partir do fluxo hardcoded do `Main`.
- `Terreno.efeitoFimDeTurno()` — inacessível a partir de `Batalha.executarTurno()` (ver achado nº 2 da tabela consolidada).
- `PokeSal.atacar()` — substituído por `Treinador.ataquePokeSal()`, nunca chamado.

1.4. Há código duplicado?

Resposta: Sim.

Justificativa: Foram identificados quatro padrões de duplicação estrutural (detalhados nos itens 9 a 12 da tabela consolidada): os seis construtores de subclasses de `PokeSal`, os métodos `usar()` de `Potion/SuperPotion`, os dois blocos simétricos de turno em `Batalha.executarTurno()` e os três ramos do switch em `Terreno.bonus()`. Todos aumentam o custo de manutenção (uma alteração de regra precisa ser replicada em múltiplos pontos).

1.5. Existem erros de sintaxe?

Resposta: Nenhum identificado na revisão manual.

Justificativa: A leitura linha a linha dos 16 arquivos não encontrou chaves, parênteses, ponto-e-vírgulas ou declarações de `import/pacote` incorretos.

Nota de complementaridade: Essa lacuna foi coberta posteriormente pela Parte 2 deste relatório. A compilação com `javac --release 21` foi executada com sucesso, confirmando a ausência de erros de sintaxe (ver Seção "Execução", Parte 2).

Recomendação: Recomenda-se compilar o projeto (via `javac` ou pela IDE/`Checkstyle` configurada via `.idea/checkstyle-idea.xml`) antes da entrega final.

1.6. Há erros de lógica que quebram regras do negócio?

Resposta: Sim.

Justificativa: Três casos concretos foram identificados (detalhados nos achados nº 5, 6 e 2 da tabela consolidada):

Evidências (arquivo / trecho):

- Dano negativo cura o oponente: em `Treinador.ataquePokeSal()`, a expressão `Math.max(1, atk*bonus)` - def resulta em valor negativo quando a defesa é maior que o ataque bonificado. Como esse valor é subtraído do HP do defensor, o defensor é curado em vez de sofrer dano —

violando a regra central de que "um ataque reduz o HP do oponente".

- Cura automática assume item fixo: `Batalha.curarSeNecessario()` sempre utiliza `itens[0]` como se fosse necessariamente uma `Potion` disponível, sem validar o conteúdo do slot.
- Regra de terreno nunca aplicada: `Terreno.efeitoFimDeTurno()` (cura de 5% do HP máximo para `PokeSal Planta` no `Canteiro Central`) nunca é chamada durante a batalha.

1.7. Há erros de tipagem?

Resposta: Nenhum identificado.

Justificativa: Não foram encontrados problemas de tipagem: conversões implícitas de `int` para `double` (ex.: `qtdCura` em `Potion/SuperPotion`) são seguras e intencionais, comparações de `enum` usam `==` corretamente e não existem casts explícitos arriscados nos arquivos.

1.8. O fluxo de controle é válido (sem loops infinitos, condições impossíveis)?

Resposta: Parcial — atenção necessária.

Justificativa: Estruturalmente, os blocos `switch` possuem cláusula `default` e os comandos `if/else` cobrem os casos de `TipoElemental` e `TipoTerreno` sem condições impossíveis. O ponto crítico é o laço `while` (vida) em `Batalha.iniciarBatalha()`: ele só termina quando o HP de um dos `PokeSal` chega a 0 via `executarTurno()`. Isso depende inteiramente do balanceamento de atributos, e não de uma garantia estrutural no código (ex.: limite de turnos).

Combinado ao achado do item 6 (dano negativo quando a defesa supera o ataque), existe um cenário teórico em que nenhum dos dois lados perde HP líquido, fazendo com que o laço nunca termine. Com os valores atuais usados em `Main.java` (`CharSal` vs. `SquirtSal`) isso não ocorre, mas o código não impede a falha em cenários com defesa alta.

Evidências (arquivo / trecho): `Batalha.java`, linhas 26–39 (`iniciarBatalha()`) — laço sem limite estrutural de iterações.

1.9. Outros tipos de erros identificados?

Resposta: Sim — observações adicionais (severidade menor).

Justificativa:

- Entrada de usuário sem tratamento: o método inativo `Batalha.escolhaItem()` usa `Integer.parseInt(sc.nextLine().trim())` sem tratar `NumberFormatException` nem validar o índice antes de chamar `usarItem(escolha - 1)`.
- Código não exercitado: seis classes/métodos completos nunca são chamados a partir do `Main`, indicando ausência de testes automatizados (confirmado na Parte 2: `src/test/java` está vazio).
- Falha silenciosa de terreno não configurado: se `Terreno.setTerreno()` não for chamado, terreno permanece `null`. Comparações como `terreno == ASFALTO_QUENTE` retornam `false` sem lançar exceção, desativando os bônus de terreno silenciosamente.

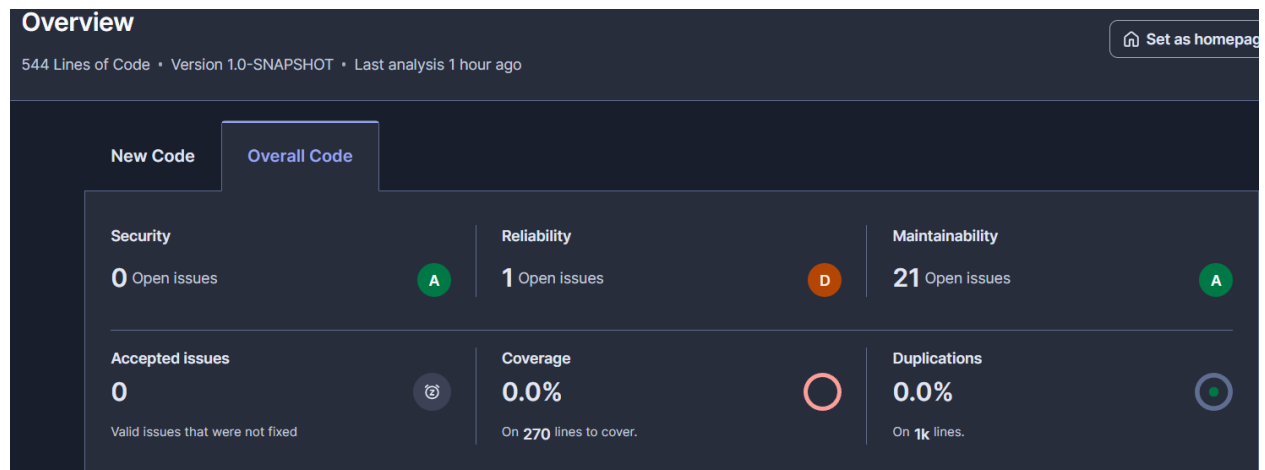
- Redundância de modificadores: em Item.java, os métodos da interface são declarados com public explícito, o que é redundante em interfaces Java.

Tabela Consolidada de Achados — Revisão Manual

#	Categoria	Local (arquivo)	Achado	Recomendação
1	Código morto	Batalha.java — escolhaItem()	Método privado nunca chamado no fluxo atual (Main usa itens fixos).	Remover o método ou integrá-lo ao fluxo interativo de batalha.
2	Código morto / Regra de negócio	Terreno.java — efeitoFimDeTurno()	Método nunca é chamado por Batalha; efeito de cura do Canteiro Central para tipo Planta não ocorre.	Chamar efeitoFimDeTurno() ao final de cada turno em Batalha.executarTurno().
3	Código morto	Pokesal.java — atacar()	Método de ataque básico nunca é usado; ataque real é feito por Treinador.ataquePokesal().	Remover se obsoleto ou documentar o propósito de permanência.
4	Código morto	SuperPotion, Antidote, BulbaSal, ChikoSal, CyndaSal, TotoSal	Classes nunca instanciadas em nenhum ponto do código.	Exercitar via testes unitários ou remover se não forem necessárias.
5	Erro de lógica (Negócio)	Treinador.java — ataquePokesal()	Cálculo Math.max(1, atk*bonus) - def gera dano negativo quando def > atk*bonus, curando o oponente.	Aplicar o piso após subtrair a defesa: Math.max(0, atk*bonus - def).
6	Erro de lógica / Exceção	Batalha.java — curarSeNecessario()	Assume que itens[0] é sempre utilizável. Se for null, lança NullPointerException não tratada.	Validar itens[0] != null e/ou tratar NullPointerException antes de usar o item.
7	Fluxo de controle	Batalha.java — iniciarBatalha()	Laço while (vida) sem limite estrutural. Se o dano for <= 0 devido ao achado nº 5, entra em loop infinito.	Corrigir o cálculo de dano e adicionar um limite máximo de turnos à batalha.
8	Robustez / Inicialização	Treinador.java (pokesal); Terreno.java (terreno)	Campos de referência iniciam como null. Uso sem setter gera NullPointerException ou falha silenciosa.	Inicializar/validar nos construtores ou lançar exceção explícita no acesso prévio.

#	Categoria	Local (arquivo)	Achado	Recomendação
9	Duplicação de código	Subclasses de Pokesal (6 arquivos)	Construtores repetem a mesma sequência de setters, alterando apenas os valores numéricos.	Criar construtor parametrizado na classe base Pokesal para reutilização.
10	Duplicação de código	Potion.usar() vs. SuperPotion.usar()	Lógica de cura idêntica; a verificação de limite máximo é redundante com Pokesal.setHp().	Padronizar a implementação delegando o limite de HP exclusivamente ao setHp().
11	Duplicação de código	Batalha.executarTurno()	Bloco "curar → atacar → checar derrota → imprimir" é duplicado para t1→t2 e t2→t1.	Extrair método auxiliar privado recebendo (atacante, defensor, terreno).
12	Duplicação de código	Terreno.bonus() — switch	Ramos do switch repete a estrutura de validação do defensor para cada tipo elemental.	Substituir por matriz/mapa de vantagens de tipo (Map<TipoElemental, Map<...>>).

2 - Análise Automatizada (SonarQube)



2.1 Objetivo

Avaliar a qualidade estática do código-fonte do projeto utilizando o SonarQube, levantando as métricas de Code Smells, Bugs, Vulnerabilities e Coverage (%).

2.2 Ambiente de Análise

Item	Valor
Projeto Analisado	projetoTestes (Simulador de Batalha – PokeSal)
Build System	Maven (pom.xml na raiz)
Chave do Projeto	Atividade-Pontuada-Testes
Servidor SonarQube	http://localhost:9000, v26.9.0.129388 (obtido em target/sonar/report-task.txt)
Scanner Declarado	sonar-maven-plugin v3.10.0.2594
Release do Compilador	Java 25 (maven.compiler.release no pom.xml)
JDK de Verificação	OpenJDK 21.0.10
Testes Automatizados	Ausentes (src/test/java sem arquivos)
Bibliotecas Externas	Nenhuma (Java Standard SDK puro + plugin Sonar)
Estrutura de Pacotes	17 arquivos .java, 952 linhas de código distribuídas nos pacotes raiz, batalha, itens, pokesal e treinador

2.3 Execução

O pacote enviado continha evidências da execução do SonarQube realizada previamente pelo autor (target/sonar/report-task.txt e target/sonar/.sonar_lock). O arquivo .txt anexado representa a exportação oficial das issues (Code Smells) dessa execução.

2.4 Resultados Mapeados

Métrica	Resultado
Code Smells	22 issues (100% do total exportado)
Bugs	0 no modelo clássico; 1 issue com impacto na dimensão Reliability

Métrica	Resultado
Vulnerabilities	0 (nenhuma issue afetando a dimensão Security)
Coverage	0% — ausência de suíte de testes automatizados

Distribuição por Severidade

- **High:** 1
- **Medium:** 20
- **Low:** 1

Distribuição por Arquivo

- batalha/Batalha.java: 15
- treinador/Treinador.java: 4
- Main.java: 2
- batalha/Terreno.java: 1

Esforço Estimado de Correção: A soma do campo effort das 21 issues exportadas totaliza 178 minutos (~2h 58min).

2.5 Problemas Encontrados (Detalhamento SonarQube)

- **Uso de System.out em vez de Logger (15 ocorrências):** Presente em Batalha.java (12) e Treinador.java (3). Representa 68% do total de issues. Classificado como má prática, impactando Adaptability e Maintainability (Severidade Medium).
- **Reinstanciação Inadequada de Random (1 ocorrência):** Em Treinador.java (L68), uma nova instância (new Random()) é criada dentro de ataquePokesal(). Como o método é chamado de forma recorrente na batalha, há degradação na qualidade da sequência pseudoaleatória e desperdício de recursos. É a única issue de severidade High e impacto em Reliability.
- **Parâmetro args não utilizado em Main.main (L21):** Impacta Intentionality/Maintainability (Severidade Medium).
- **Ordem de parâmetros inconsistente em executarTurno (Batalha.java, L34):** A ordem dos parâmetros formais difere da ordem dos argumentos passados nas chamadas (t1, t2 vs t2, t1), induzindo a erros de interpretação no fluxo.
- **Método privado não utilizado escolhaltem (Batalha.java, L89):** Identificado como código morto.
- **Caractere de quebra de linha \n em vez de %n (Batalha.java, L128):** Compromete a portabilidade entre diferentes sistemas operacionais em formatações de string.
- **Nome de campo igual ao tipo da classe (Terreno.java, L20):** O campo terreno do tipo TipoTerreno na classe Terreno reduz a clareza do código.

- **Main.java no pacote padrão:** Ausência de declaração de package, violando as convenções de estruturação Java (Severidade Low).

3 — Síntese Consolidada

A combinação das análises manual e automatizada oferece uma visão abrangente sobre o código do simulador. A revisão manual identificou falhas nas regras de negócio (cura indesejada via dano negativo, regra de terreno inativa e potencial loop infinito no combate) que não foram detectadas pelas regras padrão do SonarQube. Por outro lado, a análise via SonarQube quantificou a dívida técnica, identificou padrões de código inadequados de forma exhaustiva (ex.: System.out) e confirmou objetivamente a ausência de testes.

Priorized Backlog

1. **[Alta Prioridade / Regra de Negócio]** Corrigir a fórmula de dano em `Treinador.ataquePokesal()` para impedir valores negativos e cura do oponente: $Dano = \max(0, (Atk * Bônus) - Def)$.
2. **[Alta Prioridade / Confiabilidade - SonarQube]** Promover a instância de `Random` em `Treinador.java` a um campo final de classe (`private final Random random = new Random();`), evitando reinstituições a cada ataque.
3. **[Alta Prioridade / Cobertura]** Implementar testes unitários com JUnit cobrindo os módulos de dano, cura e terreno, configurando o plugin JaCoCo no `pom.xml` para geração de relatórios de cobertura.
4. **[Média Prioridade / Regra de Negócio]** Inserir a chamada `Terreno.efeitoFimDeTurno()` ao final de cada rodada dentro de `Batalha.executarTurno()`, ativando a cura dos `PokeSals` do tipo `Planta` no `Canteiro Central`.
5. **[Média Prioridade / Robustez]** Adicionar verificação de nulidade em `Batalha.curarSeNecessario()` para validar a existência e disponibilidade de itens no índice 0 antes do uso.
6. **[Média Prioridade / Manutenibilidade - SonarQube]** Substituir os 15 comandos `System.out.println` por um framework de log unificado (`java.util.logging` ou `SLF4J`).
7. **[Baixa Prioridade / Limpeza de Código]** Remover métodos e classes não utilizados (`escolhaltem`, `Pokesal.atacar()`, subclasses e itens não instanciados) ou incorporá-los ao fluxo principal do programa.
8. **[Baixa Prioridade / Arquitetura]** Refatorar duplicações de código (construtor parametrizado na classe base `Pokesal`, unificação da lógica de cura das `Potion/SuperPotion`, agrupamento do turno em método auxiliar e matriz de vantagens em `Terreno.bonus()`).
9. **[Baixa Prioridade / Convenções]** Mover a classe `Main.java` para um pacote nomeado, substituir `\n` por `%n` nas formatações de texto, renomear o campo `terreno` em `Terreno.java` e remover o parâmetro não utilizado `args` do método `main`.

4- Teste unitários com JUnit

4.1. Objetivo

Documentar os testes unitários existentes no projeto PokeSal, explicar como o JUnit é usado neles, registrar o resultado real de uma execução e medir a cobertura de código, apontando o que já está protegido por testes e o que ainda não está.

4.2. Conceitos de JUnit usados no projeto

Um teste unitário verifica uma pequena unidade de código (um método ou comportamento) de forma isolada e automática. O JUnit 5 é o framework Java que descobre, executa e reporta esses testes. Os recursos usados nas duas classes de teste são:

- `@Test` — marca um método como caso de teste.
- `@BeforeEach` — executa antes de cada teste, recriando o cenário (Treinador, Pokesal e Terreno novos), o que mantém os testes independentes entre si.
- `@DisplayName` — dá um nome legível ao teste no relatório de execução.
- `@ParameterizedTest` + `@ValueSource` — roda o mesmo teste com vários valores de entrada (usado em `TreinadorTest` com 5, 0 e -3).
- `assertEquals`, `assertTrue` — comparam o resultado real com o esperado.
- `assertThrows` — verifica que uma exceção específica é lançada, e permite conferir a mensagem (usado no limite de itens).

4.3. Ambiente e forma de execução

O código foi clonado do GitHub e compilado com OpenJDK 21. O Maven Central não é acessível no ambiente desta análise (retorna 403), então o comando `mvn test` não pôde ser usado. Em vez disso, os testes foram executados diretamente com o JUnit Platform Console Launcher, usando os jars do JUnit 5.10.1 disponíveis nos repositórios Ubuntu (o `pom.xml` declara 5.10.2), e a cobertura foi medida com o agente do JaCoCo 0.8.11. O resultado é equivalente para efeito de aprovação dos testes, mas não é uma execução via Maven/Surefire.

4. Testes existentes

4.1 BatalhaTest (4 testes)

Em `@BeforeEach`, os dois Pokesal têm erro, desvio e crítico zerados, eliminando a aleatoriedade e tornando os resultados determinísticos — uma boa prática para testar código com sorteios.

Teste	O que verifica	Resultado
testVantagemElemental	Ataque de CharSal (Fogo) contra SquirtSal (Água) causa dano reduzido: HP esperado do defensor 39,5.	Aprovado
testEfeitoTerrenoEstacionamentoUCSal	Com terreno ASFALTO_QUENTE o multiplicador combinado é 0,575: HP esperado 38,225.	Aprovado
testOrdemDeAtaquePorVelocidade	Com velocidade 100 x 10 e ataque 1000, o mais rápido vence sem sofrer dano (HP 39,0) e o outro chega a HP <= 0. Executa uma batalha completa.	Aprovado
testUsoLimiteDeItensExcedido	Após usar 2 poções, o terceiro uso lança IllegalStateException com a mensagem "Ronaldo já usou o limite de itens nesta batalha".	Aprovado

4.2 TreinadorTest (4 métodos, 7 execuções)

Teste	O que verifica	Resultado
numerosInvalidos (parametrizado: 5, 0, -3)	Perks inexistentes não alteram crítico, desvio nem defesa.	3 aprovados
testCalculoDanoBoundaryValues	Definir HP, DEF e ATK como 100 respeita o teto máximo de cada atributo.	Aprovado
testeCritico	Perk 1 aumenta o crítico em 10.	Aprovado
testeDesvio	Perk 2 aumenta o desvio em 5.	Aprovado
testeDefesa	Perk 3 aumenta a defesa em 5.	Aprovado

Técnicas de teste presentes: teste de valores de fronteira e classes inválidas (numerosInvalidos), teste de exceção (assertThrows), teste de regra de negócio (vantagem elemental e de terreno) e teste de integração leve (batalha completa em testOrdemDeAtaquePorVelocidade).

4.4. Resultado da execução

Métrica	Valor
Testes encontrados / iniciados	11 / 11
Aprovados	11
Falharam	0
Ignorados / abortados	0 / 0

Tempo total	247 ms
-------------	--------

4.6. Cobertura de código (JaCoCo)

Métrica	Coberto / Total	Percentual
Linhas	191 / 336	56,8%
Branches (decisões)	29 / 68	42,6%
Métodos	53 / 78	67,9%

4.6.1 Cobertura por classe (linhas)

Classe	Linhas	Branches	Comentário
pokesal.Pokesal	51/54	8/8	Praticamente completa
treinador.Treinador	37/47	8/14	Faltam listarItens, getItens, getItem
batalha.Batalha	50/82	6/12	escolhaItem (método sem uso) não coberto
batalha.Terreno	15/30	7/30	Só 7 de 30 ramos; efeitoFimDeTurno e getTerreno sem teste
itens.Potion	6/10	—	getNome, getQtdCura, setQtdCura sem teste
itens.SuperPotion, itens.Antidote	0/10, 0/8	0/4 (Antidote)	Nunca exercitadas
CharSal, SquirtSal	12/12 cada	—	Completas (únicas usadas nos testes)
BulbaSal, ChikoSal, CyndaSal, TotoSal	0/12 cada	—	Nunca exercitadas
aplication.Main	0/15	—	Ponto de entrada, sem teste

4.7. Análise crítica dos testes

4.7.1 Pontos fortes

- Cenário recriado a cada teste (@BeforeEach), sem dependência de ordem entre testes.
- Aleatoriedade neutralizada (erro, desvio e crítico em 0), deixando os testes de batalha determinísticos.
- Mensagens de falha explicativas nos assertEquals/assertTrue e nomes legíveis via @DisplayName.

- Uso de teste parametrizado e de `assertThrows` com verificação da mensagem da exceção.

4.7.2 Pontos de atenção

- Localização dos testes: eles ficam em `src/test/` e não em `src/test/java`. Pelo padrão do Maven, o diretório de testes é `src/test/java`, e o `pom.xml` não define `testSourceDirectory`; portanto, `mvn test` tende a não descobrir essas classes. O arquivo `projetoTestes.iml` marca `src/test` como pasta de testes, por isso na IDE IntelliJ eles rodam normalmente. Não pude confirmar o comportamento do Maven neste ambiente (Maven Central bloqueado).
- Sem JaCoCo no `pom.xml`: o SonarQube lê relatórios de cobertura, e o `pom` não configura nenhum gerador. Mesmo com os testes rodando, a métrica Coverage do Sonar continuaria em 0% até o JaCoCo ser adicionado (o valor de 56,8% acima foi medido por fora do Maven).
- Nome enganoso: `testCalculoDanoBoundaryValues` não testa cálculo de dano; ele verifica que `setHp`, `setDef` e `setAtk` respeitam os valores máximos.
- Comparação de doubles sem tolerância: `assertEquals(38.225, vidaAtual)` compara exatamente; funciona hoje, mas é frágil a mudanças na ordem das operações. O JUnit permite informar um delta (`assertEquals(esperado, atual, 0.001)`).
- Ruído no console: a execução imprime a narração da batalha (`System.out` em `Batalha`), poluindo a saída dos testes.
- Pacote da classe `Main`: o arquivo está em `src/main/java/application/`, mas declara `package aplicacion` (sem o segundo “p”). Compila, mas é inconsistente e explica a pasta `target/classes/aplicacion`.

4.8. Lacunas de cobertura e testes sugeridos

Com base nos métodos não cobertos e nos achados do checklist de teste estático, os testes mais valiosos a acrescentar são:

- `Terreno.bonus`: cobrir os demais terrenos e as combinações de tipos (só 7 de 30 branches estão cobertos).
- Dano quando a defesa é maior que o ataque bonificado: teste de fronteira para garantir que o dano nunca seja negativo (relacionado ao achado nº 5 do checklist).
- `Terreno.efeitoFimDeTurno`: nunca exercitado (achado nº 2 do checklist).
- Batalha com `itens[0]` nulo (cura automática sem item): teste com `assertThrows` ou tratamento esperado (achado nº 6 do checklist).
- `SuperPotion` e `Antidote`: `usar()` com HP cheio, HP baixo e efeito do antídoto.
- Treinador: `getItens`, `getItem` e `listarItens`; `Potion`: `getters/setters`.
- Os quatro Pokesal não usados (`BulbaSal`, `ChikoSal`, `CyndaSal`, `TotoSal`): um teste parametrizado com os atributos iniciais de cada um.

4.9. Como configurar para o Maven e o SonarQube

Para que mvn test encontre os testes e o Sonar receba a cobertura, o caminho mais simples é mover as classes de teste para src/test/java (mantendo os pacotes batalha e treinador) e adicionar o plugin do JaCoCo ao pom.xml. Trecho de referência, a ser aplicado pela equipe:

```
<plugin> <groupId>org.jacoco</groupId> <artifactId>jacoco-maven-plugin</artifactId> <version>0.8.12</version>
<executions> <execution><goals><goal>prepare-agent</goal></goals></execution>
<execution><id>report</id><phase>test</phase><goals><goal>report</goal></goals></execution> </executions>
</plugin>
```

Depois: mvn clean verify sonar:sonar -Dsonar.projectKey=Atividade-Pontuada-Testes
-Dsonar.host.url=http://localhost:9000 -Dsonar.token=<seu_token>. O Sonar passará a mostrar a métrica Coverage com base nos testes.

4.10. Conclusão

O projeto passou de zero testes automatizados (situação registrada no relatório do SonarQube) para 11 execuções de teste JUnit, todas aprovadas, cobrindo 56,8% das linhas e 42,6% dos branches. Os testes existentes são bem estruturados — cenário isolado, aleatoriedade controlada, teste parametrizado e teste de exceção — e cobrem bem a classe Pokesal e o fluxo principal de Batalha. As maiores lacunas estão em Terreno (regras de terreno), nos itens SuperPotion/Antidote e nos quatro Pokesal ainda não usados. Antes da entrega, vale ajustar a localização dos testes e configurar o JaCoCo, para que Maven e SonarQube reconheçam os testes e reportem a cobertura real.

5 - Relatório de Contribuição Individual

Eduardo Dragone Pinheiro Corrêa

Refatoração do sistema e testes com SonarQube

Atuou na refatoração do código do sistema, buscando melhorar sua organização, legibilidade e manutenção. Também participou da realização e análise de testes utilizando o SonarQube, contribuindo para a identificação de problemas e melhorias na qualidade do código.

Artur de Almeida Ramos

Testes com JUnit e refatoração do código

Atuou na refatoração do código do sistema, contribuindo para a melhoria de sua estrutura e organização. Também participou da criação e execução de testes automatizados com JUnit, auxiliando na verificação do funcionamento das funcionalidades e na identificação de possíveis falhas.

Miguel Antonio Torres Diogo

Refatoração do sistema e testes manuais e com JUnit

Participou da refatoração do sistema, contribuindo para a organização e melhoria da estrutura do código. Também atuou na realização de testes manuais e automatizados com JUnit,

verificando o comportamento das funcionalidades e auxiliando na identificação de erros durante o desenvolvimento.

Anthony Davidson Bitencourt Moura de Araújo

Testes manuais

Atuou na realização de testes manuais do sistema, verificando o funcionamento das principais funcionalidades e identificando possíveis erros ou comportamentos inesperados. Também contribuiu para o registro dos resultados dos testes e para a validação das funcionalidades implementadas.